

Train your own LLMs - SFT using QLoRA and PEFT

Zijin CAI - Student ID: 224040002
The Chinese University of Hong Kong, Shenzhen
Shenzhen, Guangdong, China
Email: 224040002@link.cuhk.edu.cn

Abstract

This study focuses on a carefully designed instruction fine-tuning large language model (LLM) for Chinese medical dialogue generation using the QLoRA technique. Leveraging the Huatuo26M-Lite dataset, the implementation adapts the Qwen-7B-Instruct model to improve its domain-specific response quality. The experiments emphasize data engineering, parameter-efficient training, and systematic evaluation. Results demonstrate a 80% accuracy on a multiple-choice medical QA benchmark, highlighting the model capability to generate contextually appropriate answers. This work provides insights into efficient LLM adaptation for specialized domains with limited computational resources.

1 Introduction

The rapid advancement of large language models (LLMs) has opened new possibilities for domain-specific applications, yet adapting general-purpose models to specialized fields like healthcare remains challenging. This research focuses on fine-tuning LLMs for Chinese medical dialogue generation, a critical task given the growing demand for accessible and reliable medical information in Chinese-speaking communities. Medical question answering (QA) systems require not only linguistic fluency but also domain expertise to provide accurate and safe recommendations, making this task both socially impactful and technically complex.

To address these challenges, this report fine-tuned the Qwen-7B-Instruct model using QLoRA, a parameter-efficient method, on the Huatuo26M-Lite dataset containing doctor-patient conversations. By leveraging 4-bit quantization and Low-Rank Adaptation (LoRA), this approach optimized computational efficiency while preserving model performance under GPU memory constraints (16GB requirement). The fine-tuned model achieved 80% accuracy on a medical multiple-choice QA benchmark, demonstrating significant improvements in domain relevance compared to the baseline. Additionally, it generated clinically coherent responses to real-world patient queries, such as advising hydration and rest for throat discomfort caused by excessive snacking. This work highlights the feasibility of adapting LLMs to specialized domains with limited resources while maintaining practical utility on the task of Chinese medical dialogue generation.

2 Experiment Design

The task involves developing a Chinese medical dialogue assistant capable of understanding patient inquiries and generating clinically relevant responses, with additional competency in solving domain-specific multiple-choice questions. To achieve this, the experiment was structured into the following four key phases, balancing computational efficiency, data quality, and task relevance to optimize the model practical deployment potential.

- data engineering: focused on curating and preprocessing the Huatuo26M-Lite dataset, transforming raw doctor-patient dialogues into structured instruction-response pairs to align with the model conversational format. The dataset was preprocessed to extract relevant question-answer pairs, then load and transform the data into a suitable format for training. The following code snippet illustrates the data preprocessing steps:

```
from datasets import load_dataset
from torch.utils.data import random_split
import transformers
from typing import Dict, Sequence, List
from torch.utils.data import Dataset
from dataclasses import dataclass

dataset = load_dataset("FreedomIntelligence/Huatuo26M-Lite")
dataset = dataset['train'].map(lambda sample: {"conversations":
    [{"from": "human", "value": sample['question']},
    {"from": "gpt", "value": sample['answer']}]}, batched=False)

train_dataset_size, val_dataset_size = 40, 8
train_dataset, val_dataset, _ = random_split(dataset,
    [train_dataset_size, val_dataset_size,
    len(dataset)-train_dataset_size-val_dataset_size])

def preprocess(
    sources,
    tokenizer: transformers.PreTrainedTokenizer,
) -> Dict:
    template = Template(tokenizer.chat_template)
    max_seq_len = tokenizer.model_max_length
    messages = []
    for i, source in enumerate(sources):
        if source[0]["from"] != "human":
            source = source[1:]

        for j in range(0, len(source), 2):
            if j+1 >= len(source): continue
            q = source[j]["value"]
            a = source[j+1]["value"]
            assert q is not None and a is not None, f'q:{q} a:{a}'
            input = template.render(messages=[{"role": "user", "content": q},
                {"role": "assistant", "content": a}],
                bos_token=tokenizer.bos_token, add_generation_prompt=False)
            input_ids = tokenizer.encode(input, add_special_tokens=False)

            query = template.render(messages=[{"role": "user", "content": q}],
                bos_token=tokenizer.bos_token, add_generation_prompt=True)
            query_ids = tokenizer.encode(query, add_special_tokens=False)

            labels = [-100]*len(query_ids) + input_ids[len(query_ids):]
            assert len(labels) == len(input_ids)
            if len(input_ids) == 0: continue
            messages.append({"input_ids": input_ids[-max_seq_len:],
                "labels": labels[-max_seq_len:]})

    input_ids = [item["input_ids"] for item in messages]
    labels = [item["labels"] for item in messages]
    max_len = max(len(x) for x in input_ids)
    max_len = min(max_len, max_seq_len)
    input_ids = [ item[:max_len] + [tokenizer.eos_token_id]
        *(max_len-len(item)) for item in input_ids]
    labels = [ item[:max_len] + [-100]*(max_len-len(item)) for item in labels]

    input_ids = torch.LongTensor(input_ids)
    labels = torch.LongTensor(labels)
    return {
        "input_ids": input_ids,
        "labels": labels
    }
```

```
class InstructDataset(Dataset):
    def __init__(self, data: Sequence,
                  tokenizer: transformers.PreTrainedTokenizer) -> None:
        super().__init__()
        self.tokenizer = tokenizer
        self.data = data

    def __len__(self):
        return len(self.data)

    def __getitem__(self, index) -> Dict[str, torch.Tensor]:
        sources = self.data[index]
        if isinstance(index, int):
            sources = [sources]
        data_dict = preprocess([e['conversations'] for e in sources],
                               self.tokenizer)
        if isinstance(index, int):
            data_dict = dict(input_ids=data_dict["input_ids"][0],
                            labels=data_dict["labels"][0])
        return data_dict

@dataclass
class DataCollatorForSupervisedDataset(object):
    tokenizer: transformers.PreTrainedTokenizer
    def __call__(self, instances: Sequence[Dict]) -> Dict[str, torch.Tensor]:
        input_ids, labels = tuple([instance[key] for instance in instances]
                                   for key in ("input_ids", "labels"))
        input_ids = torch.nn.utils.rnn.pad_sequence(
            input_ids,
            batch_first=True,
            padding_value=self.tokenizer.pad_token_id)
        labels = torch.nn.utils.rnn.pad_sequence(labels,
                                                  batch_first=True, padding_value=IGNORE_INDEX)
        return dict(
            input_ids=input_ids,
            labels=labels,
            attention_mask=input_ids.ne(self.tokenizer.pad_token_id),
        )

train_dataset = InstructDataset(train_dataset, tokenizer)
val_dataset = InstructDataset(val_dataset, tokenizer)
data_collator = DataCollatorForSupervisedDataset(tokenizer=tokenizer)
```

- model selection: prioritized Qwen-7B-Instruct for its proficiency in Chinese language tasks and instruction-following capabilities, as demonstrated in the following code snippet:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
model_id = "Qwen/Qwen2.5-7B-Instruct"

# Activate nested quantization for 4-bit base models (double quantization)
# Quantization type (fp4 or nf4), According to QLoRA paper,
# for training 4-bit base models (e.g. using LoRA adapters) one should use
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)
model = AutoModelForCausalLM.from_pretrained(model_id,
                                              quantization_config=bnb_config, device_map={"":0})

tokenizer = AutoTokenizer.from_pretrained(model_id)
```

- training methodology: employed QLoRA ($r = 8$, $\text{lora_alpha} = 8$) with 4-bit quantization to enable parameter-efficient fine-tuning under GPU memory constraints, using a cosine learning rate scheduler over custom epochs with a given batch size.

```
from peft import prepare_model_for_kbit_training
from peft import LoraConfig, get_peft_model
from jinja2 import Template

model.gradient_checkpointing_enable()
model = prepare_model_for_kbit_training(model)

# You can try different parameter-efficient strategy for model training,
# for more info, please check https://github.com/huggingface/peft
config = LoraConfig(
    r=8,
    lora_alpha=8,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)

model = get_peft_model(model, config)
template = Template(tokenizer.chat_template)

# Set training parameters
training_arguments = transformers.TrainingArguments(
    output_dir="./checkpoints",
    num_train_epochs=50,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=1,
    optim='paged_adamw_8bit',
    save_steps=100,
    logging_steps=10,
    learning_rate=1e-5,
    weight_decay=0.001,
    max_steps=-1,
    warmup_ratio=0.1,
    group_by_length=True,
    lr_scheduler_type="cosine",
    gradient_checkpointing=True,
    report_to="none"
)

model.train()
trainer = transformers.Trainer(
    model=model,
    tokenizer=tokenizer,
    args=training_arguments,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    data_collator=data_collator
)
trainer.train()
```

- evaluation: combined perplexity measurements on validation data and task-specific accuracy testing on a medical QA benchmark (1. exam. json) to assess both language coherence and domain expertise. The following code snippet illustrates the evaluation process:

```
import math
!pip install -q -U git+https://github.com/huggingface/accelerate.git
eval_results = trainer.evaluate()
print(f"Perplexity: {math.exp(eval_results['eval_loss']):.2f}")
```

3 Implementation and Results

3.1 Evaluation Setup

The evaluation framework encompassed two primary metrics: perplexity to measure language modeling performance and task-specific accuracy for domain competency. Validation perplexity was calculated on samples loss from the Huatuo26M-Lite dataset, while accuracy was tested on a medical QA benchmark (1.exam.json) containing 100 multiple-choice questions. The baseline model, the original Qwen-7B-Instruct without fine-tuning, served as a reference for performance comparison. Experiments were conducted on both Google Colab and local environments, using PyTorch with 4-bit quantization to accommodate memory constraints. To ensure fairness, the same template and decoding parameters were applied across all evaluations.

```
import json
import re
from tqdm import tqdm
from jinja2 import Template
from peft import PeftModel
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from transformers import BitsAndBytesConfig, AutoConfig
import transformers

# Load the model and tokenizer
model_id = "Qwen/Qwen2.5-7B-Instruct"
output_path = "ilora"

model = AutoModelForCausalLM.from_pretrained(model_id,
                                              load_in_8bit=True, device_map={"":0})
model = PeftModel.from_pretrained(model, output_path)
model = model.merge_and_unload()
model.config.max_length = 512
model.eval()

tokenizer = transformers.AutoTokenizer.from_pretrained(model_id, padding_side="left")
template = Template(tokenizer.chat_template)

@torch.no_grad()
def generate(prompts, batch_size=4):
    generated_texts = []
    for i in range(0, len(prompts), batch_size):
        batch_prompts = prompts[i:i + batch_size]
        model_inputs = [
            template.render(
                messages=[{"role": "user", "content": prompt}],
                bos_token=tokenizer.bos_token,
                add_generation_prompt=True
            )
            for prompt in batch_prompts
        ]
        input_ids = tokenizer(
            model_inputs,
            add_special_tokens=False,
            return_tensors='pt',
            padding=True
        ).to("cuda:0")

        outputs = model.generate(
            input_ids=input_ids,
            attention_mask=input_ids.attention_mask,
            max_new_tokens=100
        )
```

```
for j in range(len(batch_prompts)):
    generated_ids = outputs[j, input_ids.input_ids.shape[1]:]
    generated_text = tokenizer.decode(
        generated_ids,
        skip_special_tokens=True,
        clean_up_tokenization_spaces=True
    )
    generated_texts.append(generated_text)

return generated_texts

with open('1.exam.json') as f:
    data = json.load(f)

your_prompt = """
你是一个药剂师考试能手，下面是一道{question_type}，
请先详细分析问题：{question}，
然后从以下选项中选择正确答案：{options}
请直接正确答案选项，不要输出其他内容。"""

def get_query(da):
    da['options'] = '\n'.join([f"{k}:{v}" for k, v in da['option'].items() if v])
    return your_prompt.format_map(da)

for item in data:
    item['query'] = get_query(item)

model_answers = generate([item['query'] for item in data])

def get_ans(ans):
    """
    从模型生成的答案中提取选项（如 A, B, C, D, E）。
    支持多种格式的答案提取。
    """
    # 匹配单个选项（A-E），包括可能的分隔符
    match = re.findall(r'[A-E]', ans)
    # 去重并保持顺序
    return ''.join(sorted(set(match), key=match.index))

correct_num = 0
total_num = 0
for model_answer, item in tqdm(zip(model_answers, data)):
    if get_ans(model_answer) == item['answer']:
        correct_num += 1
    total_num += 1
    item['model_answer'] = model_answer

print(f"ACC: {correct_num/total_num:.2%}")

result_path = "result.json"
with open(result_path, "w", encoding="utf-8") as file:
    json.dump(data, file, ensure_ascii=False, indent=4)
    print(f"Results are save in {result_path}")
```

3.2 Quantitative Results

The fine-tuned model achieved a validation perplexity of 5.31, which is significantly lower than the baseline perplexity (>10), indicating improved language coherence in medical contexts. On the QA benchmark, the model attained 80% accuracy (80/100 correct answers), outperforming the baseline random-guessing performance. Notably, the model demonstrated robustness in handling common symptoms (e.g., sore throat, insomnia) but struggled with rare medical conditions, reflecting limitations in dataset coverage. Training efficiency was maintained with a peak GPU memory usage of 12.76GB, validating the effectiveness of QLoRA for resource-constrained adaptation.

Query Example: “我喉咙不太舒服，吃瓜子吃多了，我该怎么办？”

query:

我喉咙不太舒服，吃瓜子吃多了，我该怎么办

model_return_string:

< |im_start| >system

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.< |im_end| >

< |im_start| >user

我喉咙不太舒服，吃瓜子吃多了，我该怎么办< |im_end| >

< |im_start| >assistant

如果是因为吃太多瓜子导致喉咙不适，可以采取以下措施：

1. 多喝水，保持咽喉湿润，缓解干燥疼痛，温盐水还可辅助消炎
2. 停止食用瓜子，避免进一步刺激咽喉黏膜，防止划伤加重
3. 含服润喉片/含片，如西瓜霜含片、金嗓子喉宝等，可暂时缓解疼痛和炎症
4. 如果症状持续或加重，建议就医检查，排除其他潜在问题

response:

如果是因为吃太多瓜子导致喉咙不适，可以采取以下措施：

1. 多喝水，保持咽喉湿润，缓解干燥疼痛，温盐水还可辅助消炎
2. 停止食用瓜子，避免进一步刺激咽喉黏膜，防止划伤加重
3. 含服润喉片/含片，如西瓜霜含片、金嗓子喉宝等，可暂时缓解疼痛和炎症
4. 如果症状持续或加重，建议就医检查，排除其他潜在问题

3.3 Case study

A successful example involved query: “我喉咙不太舒服，吃瓜子吃多了，我该怎么办” (“My throat is uncomfortable from eating too many sunflower seeds. What should I do?”). The model generated a structured response advising hydration, avoiding irritants, and rest, aligning with standard medical guidelines. In contrast, a failure case emerged when addressing a question about opioid dosage limits: the model hallucinated incorrect regulatory thresholds, likely due to insufficient training data on pharmaceutical regulations. Another limitation surfaced in handling multi-step diagnostic reasoning, where the model provided fragmented advice instead of a cohesive workflow. These cases underscore the trade-off between parameter efficiency and nuanced domain expertise.

4 Conclusion

This study successfully adapted the Qwen-7B-Instruct model for Chinese medical dialogue generation through QLoRA-based fine-tuning, demonstrating that parameter-efficient methods can effectively bridge general-purpose LLMs with specialized domains under resource constraints. The experiment revealed the critical importance of high-quality domain-specific data, as evidenced by the model’s strong performance on common medical queries but limitations in handling rare conditions and precise pharmaceutical knowledge. While the 4-bit quantization and LoRA adapters enabled practical implementation on limited GPU resources, the 80% accuracy on medical QA benchmarks suggests room for improvement in capturing nuanced clinical reasoning. The case studies highlighted both the model’s ability to generate clinically sound advice for routine symptoms and its occasional tendency to hallucinate on complex or data-scarce topics. These findings underscore the need for balanced approaches that combine computational efficiency with sufficient domain coverage, pointing to future directions such as hybrid dataset construction and enhanced safety mechanisms for medical applications. The work provides a practical framework for deploying specialized LLMs in healthcare while emphasizing challenges in achieving domain expertise.

Acknowledgment

Assignment3 for NLP Course, see details in <https://nlp-course-cuhksz.github.io/>.

Google Colab Notebook: https://colab.research.google.com/drive/1AP5kIzo_BKqfmmY8pl4l2kkNec8zkMoh?usp=sharing

Tutorial 5 Materials for LLM Training: <https://docs.google.com/presentation/d/1v9p3N0Qn6g0LcaGeUjQwLRp-egoy8ZPH/edit#slide=id.p41>

LaTeX Template for NLP Course Assignment3: <https://www.overleaf.com/project/67e76657139983ebd8f0988d>

FreedomIntelligence/Huatuo-26M HuggingFace: <https://huggingface.co/datasets/FreedomIntelligence/Huatuo26M-Lite>

HuggingFace Transformers: <https://huggingface.co/docs/transformers/index>

HuggingFace PEFT: <https://huggingface.co/docs/peft/index>

HuggingFace Accelerate: <https://huggingface.co/docs/accelerate/index>

PyTorch CUDA Environment Variables: <https://pytorch.org/docs/stable/notes/cuda.html#cuda-environment-variables>